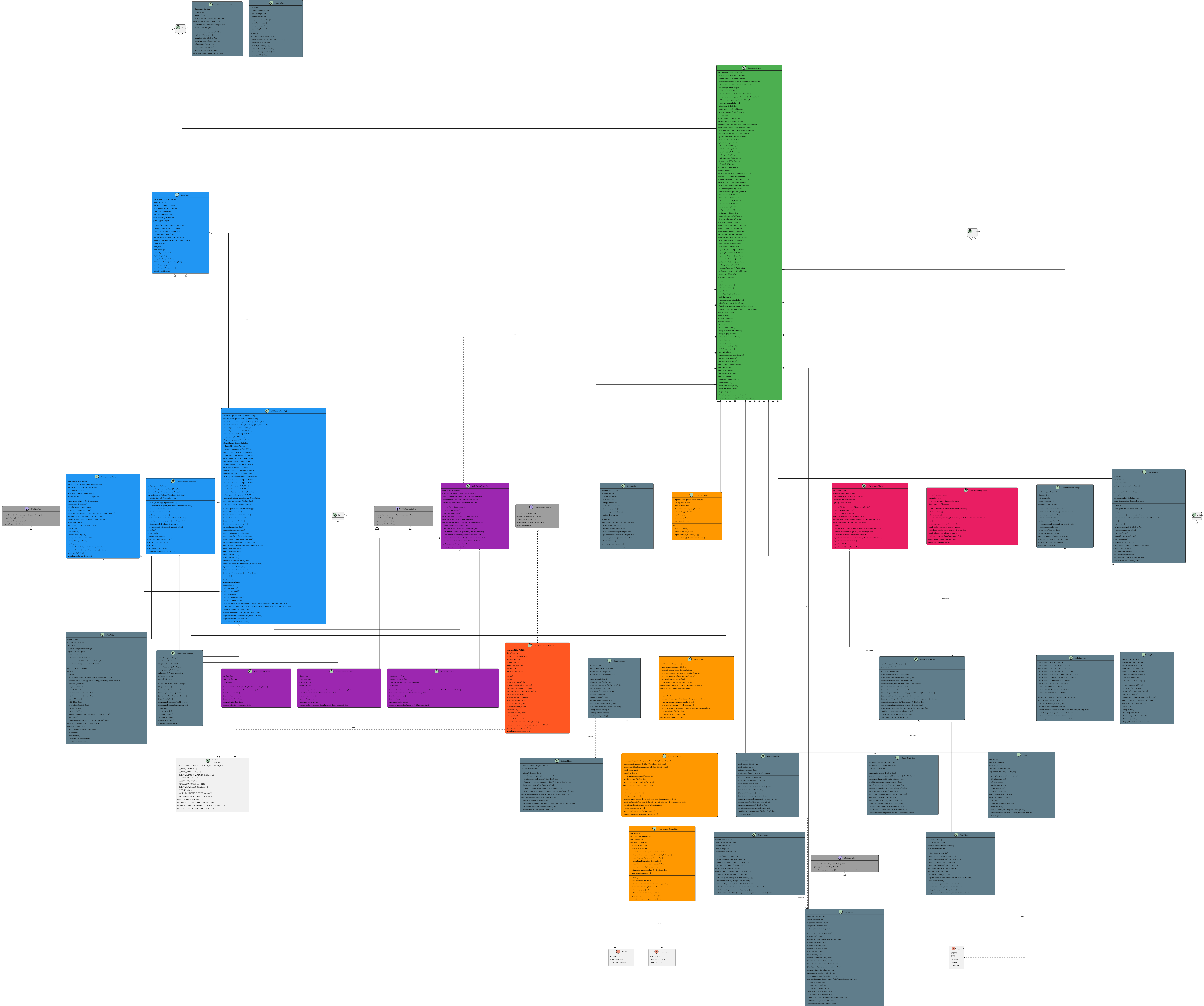
MeasurementMetadata
+timestamp: datetime
+operator: str
+sample_id: str
+measurement_conditions: Dict[str, Any]
+instrument_settings: Dict[str, Any]
+environmental_conditions: Dict[str, float]
+quality_flags: List[str]
+__init__(operator: str, sample_id: str)
+to_dict(): Dict[str, Any]
+from_dict(data: Dict[str, Any])
+export_metadata(format: str): str
+validate_metadata(): bool
+add_quality_flag(flag: str)
+remove_quality_flag(flag: str)
+get_measurement_duration(): timedelta
QualityReport
+snr: float
+baseline_stability: bool
+peak_quality: float
+overall_score: float
+recommendations: List[str]
+error_flags: List[str]
+timestamp: datetime
+data_integrity: bool
+__init__()
+calculate_overall_score(): float
+add_recommendation(recommendation: str)
+add_error_flag(flag: str)
+to_dict(): Dict[str, Any]
+from_dict(data: Dict[str, Any])
+export_report(format: str): str
+is_acceptable(): bool
QWidget
BasePanel
parent_app: SpectrometerApp
is_dark_theme: bool
left_column_widget: QWidget
right_column_widget: QWidget
main_splitter: QSplitter
left_layout: QVBoxLayout
right_layout: QVBoxLayout
panel_logger: Logger
+__init__(parent_app: SpectrometerApp)
+on_theme_changed(is_dark: bool)
+resizeEvent(event: QResizeEvent)
+validate_panel_state(): bool
+export_panel_settings(): Dict[str, Any]
+import_panel_settings(settings: Dict[str, Any])
_setup_base_ui()
_init_plots()
_init_controls()
_connect_panel_signals()
_log(message: str)
_get_plot_colors(): Dict[str, str]
_handle_panel_error(error: Exception)
signal+logMessage(str)
signal+requestMeasurement()
signal+panelError(str)
SpectrometerApp
-plot_options: PlotOptionsState
-data_state: MeasurementDataState
-calibration_state: CalibrationState
-measurement_control_state: MeasurementControlState
-calculation_controller: CalculationController
-file_manager: FileManager
-serial_worker: SerialWorker
-main_spectrum_panel: MainSpectrumPanel
-concentration_curve_panel: ConcentrationCurvePanel
-calibration_curve_tab: CalibrationCurveTab
-current_theme_is_dark: bool
-help_dialog: HelpDialog
-config_manager: ConfigManager
-session_manager: SessionManager
-logger: Logger
-error_handler: ErrorHandler
-backup_manager: BackupManager
-communication_manager: CommunicationManager
-measurement_thread: MeasurementThread
-data_processing_thread: DataProcessingThread
-statistics_calculator: StatisticsCalculator
-quality_controller: QualityController
-data_validator: DataValidator
-system_info: SystemInfo
-tab_widget: QTabWidget
-central_widget: QWidget
-main_layout: QVBoxLayout
-control_panel: QWidget
-control_layout: QHBoxLayout
-right_layout: QVBoxLayout
-left_panel: QWidget
-left_layout: QVBoxLayout
-splitter: QSplitter
-measurement_group: CollapsibleGroupBox
-display_group: CollapsibleGroupBox
-calibration_group: CollapsibleGroupBox
-buttons_group: CollapsibleGroupBox
-measurement_type_combo: QComboBox
-m_samples_spinbox: QSpinBox
-p_measurements_spinbox: QSpinBox
-start_button: QPushButton
-stop_button: QPushButton
-calculate_button: QPushButton
-reset_button: QPushButton
-epsilon_input: QLineEdit
-path_length_input: QLineEdit
-port_combo: QComboBox
-connect_button: QPushButton
-disconnect_button: QPushButton
-log_scale_checkbox: QCheckBox
-show_markers_checkbox: QCheckBox
-show_fit_checkbox: QCheckBox
-superimpose_combo: QComboBox
-plot_type_combo: QComboBox
-subtract_blank_checkbox: QCheckBox
-reset_blank_button: QPushButton
-theme_button: QPushButton
-help_button: QPushButton
-export_log_button: QPushButton
-export_plot_button: QPushButton
-export_csv_button: QPushButton
-save_session_button: QPushButton
-load_session_button: QPushButton
-backup_button: QPushButton
-system_info_button: QPushButton
-quality_report_button: QPushButton
-status_bar: QStatusBar
-log_text: QTextEdit
+__init__()
+start_measurement()
+stop_measurement()
+update_ui()
+handle_serial_data(data: str)
+switch_theme()
+on_theme_changed(is_dark: bool)
+closeEvent(event: QCloseEvent)
+handle_measurement_complete(data: ndarray)
+handle_quality_assessment(report: QualityReport)
+show_system_info()
+create_backup()
+load_configuration()
+save_configuration()
-_setup_ui()
-_setup_control_panel()
-_setup_measurement_controls()
-_setup_display_controls()
-_setup_calibration_controls()
-_setup_buttons()
-_connect_signals()
-_connect_thread_signals()
-_initialize_managers()
-_setup_logging()
-_on_measurement_type_changed()
-_on_start_measurement()
-_on_stop_measurement()
-_on_calculate_concentration()
-_on_reset_blank()
-_on_connect_serial()
-_on_disconnect_serial()
-_on_port_refresh()
-_update_superimpose_list()
-_update_ui_state()
-_show_error(message: str)
-_show_info(message: str)
-_log(message: str)
-_handle_critical_error(error: Exception)
-_validate_measurement_data(data: ndarray): bool
QThread
SerialWorker
-port: str
-baudrate: int
-is_running: bool
-serial_connection: Optional[Serial]
-data_queue: Queue
-communication_timeout: float
-retry_attempts: int
-protocol_handler: SerialProtocol
-connection_monitor: ConnectionMonitor
+__init__()
+start(port: str, baudrate: int): bool
+stop()
+send_command(command: str): bool
+send_command_with_response(command: str): Optional[str]
+run()
+is_connected(): bool
+get_connection_info(): Dict[str, Any]
+test_communication(): bool
+reconnect(): bool
-_establish_connection(): bool
-_read_serial_data()
-_parse_serial_data(data: str)
-_handle_communication_error(error: Exception)
-_monitor_connection()
signal+dataReceived(str)
signal+errorOccurred(str)
signal+connectionStatusChanged(bool)
signal+deviceInfoReceived(Dict)
CommunicationManager
-protocol: SerialProtocol
-timeout: float
-retry_count: int
-connection_status: bool
-command_queue: Queue
+__init__(protocol: SerialProtocol)
+send_command_with_retry(command: str): str
+establish_connection(): bool
+test_communication(): bool
+get_connection_status(): bool
+queue_command(command: str, priority: int)
+process_command_queue()
+set_timeout(timeout: float)
+set_retry_count(count: int)
-_execute_command(command: str): str
-_validate_response(response: str): bool
-_handle_communication_timeout()
-_prioritize_commands()
MeasurementThread
-is_running: bool
-measurement_queue: Queue
-device_interface: IMeasurementDevice
-measurement_interval: float
-quality_threshold: float
+__init__(device_interface: IMeasurementDevice)
+start_measurement_loop()
+stop_measurement_loop()
+set_measurement_interval(interval: float)
+queue_measurement_request(request: MeasurementRequest)
+get_measurement_status(): Dict[str, Any]
+run()
-_process_measurement_request(request: MeasurementRequest)
-_validate_measurement_quality(data: ndarray): bool
-_handle_measurement_error(error: Exception)
signal+measurementComplete(ndarray, MeasurementMetadata)
signal+measurementError(str)
signal+qualityAlert(str)
signal+deviceStatusChanged(Dict)
DataProcessingThread
-processing_queue: Queue
-is_running: bool
-statistics_calculator: StatisticsCalculator
-filter_manager: FilterManager
+__init__(statistics_calculator: StatisticsCalculator)
+start_processing()
+stop_processing()
+queue_data_for_processing(data: ndarray, metadata: MeasurementMetadata)
+run()
-_process_raw_data(raw_data: str): ndarray
-_apply_calibration(data: ndarray): ndarray
-_calculate_statistics(data: ndarray): Dict[str, float]
-_apply_filters(data: ndarray): ndarray
-_validate_processed_data(data: ndarray): bool
signal+dataProcessed(ndarray, Dict)
signal+processingError(str)
QDialog
CalibrationCurveTab
-calibration_points: List[Tuple[float, float]]
-transfer_model_points: List[Tuple[float, float]]
-fit_result_abs_vs_conc: Optional[Tuple[float, float, float]]
-fit_result_transfer_model: Optional[Tuple[float, float, float]]
-plot_widget_abs_vs_conc: PlotWidget
-plot_widget_transfer_model: PlotWidget
-cal_wavelength_combo: QComboBox
-conc_input: QDoubleSpinBox
-abs_custom_input: QDoubleSpinBox
-abs_ref_input: QDoubleSpinBox
-points_table: QTableWidget
-transfer_points_table: QTableWidget
-add_calibration_button: QPushButton
-remove_calibration_button: QPushButton
-clear_calibration_button: QPushButton
-add_transfer_button: QPushButton
-remove_transfer_button: QPushButton
-clear_transfer_button: QPushButton
-apply_calibration_button: QPushButton
-apply_transfer_button: QPushButton
-clear_applied_transfer_button: QPushButton
-load_calibration_button: QPushButton
-save_calibration_button: QPushButton
-load_transfer_button: QPushButton
-save_transfer_button: QPushButton
-measure_abs_custom_button: QPushButton
-validate_calibration_button: QPushButton
-export_calibration_report_button: QPushButton
-calibration_uncertainty: Dict[int, float]
-residuals_analysis: Optional[ndarray]
+__init__(parent_app: SpectrometerApp)
+add_calibration_point()
+remove_selected_calibration_point()
+clear_all_calibration_points()
+add_transfer_model_point()
+remove_selected_transfer_point()
+clear_all_transfer_points()
+update_table_and_plot_all()
+apply_calibration_to_main_app()
+apply_transfer_model_to_main_app()
+clear_transfer_model_from_main_app()
+request_direct_absorbance_measurement()
+handle_direct_measurement_result(absorbance: float)
+load_calibration_data()
+save_calibration_data()
+load_transfer_data()
+save_transfer_data()
+validate_calibration_curve(): bool
+calculate_calibration_uncertainty(): Dict[int, float]
+perform_residuals_analysis(): ndarray
+generate_calibration_report(): str
+export_calibration_report(format: str): bool
_init_plots()
_init_controls()
_connect_panel_signals()
-_calculate_fits()
-_plot_abs_vs_conc()
-_plot_transfer_model()
-_plot_residuals()
-_update_calibration_table()
-_update_transfer_table()
-_perform_linear_regression(x_data: ndarray, y_data: ndarray): Tuple[float, float, float]
-_calculate_r_squared(x_data: ndarray, y_data: ndarray, slope: float, intercept: float): float
-_validate_calibration_points(): bool
signal+calibrationApplied(int, float, float, float)
signal+transferModelApplied(int, float, float, float)
signal+transferModelCleared()
signal+calibrationValidated(bool)
MainSpectrumPanel
-plot_widget: PlotWidget
-measurement_controls: CollapsibleGroupBox
-display_controls: CollapsibleGroupBox
-wavelengths: ndarray
-spectrum_renderer: IPlotRenderer
-current_spectrum_data: Optional[ndarray]
+__init__(parent_app: SpectrometerApp)
+update_spectrum_plot()
+handle_measurement_request()
+clear_superimposed_spectra()
+add_spectrum_to_superimpose(label: str, spectrum: ndarray)
+export_current_spectrum(format: str): bool
+zoom_to_wavelength_range(start: float, end: float)
+reset_plot_view()
+apply_smoothing_filter(filter_type: str)
_init_plots()
_init_controls()
_connect_panel_signals()
-_setup_measurement_controls()
-_setup_display_controls()
-_plot_spectrum()
-_get_spectrum_data(): Tuple[ndarray, ndarray]
-_convert_to_plot_type(spectrum: ndarray): ndarray
-_apply_plot_styling()
-_handle_plot_interaction(event)
ConcentrationCurvePanel
-plot_widget: PlotWidget
-concentration_points: List[Tuple[float, float]]
-concentration_controls: CollapsibleGroupBox
-curve_fit_result: Optional[Tuple[float, float, float]]
-prediction_interval: Optional[ndarray]
+__init__(parent_app: SpectrometerApp)
+add_concentration_point(time: float, concentration: float)
+remove_concentration_point(index: int)
+clear_concentration_points()
+update_concentration_plot()
+fit_concentration_curve(): Tuple[float, float, float]
+predict_concentration_at_time(time: float): float
+calculate_prediction_interval(): ndarray
+export_concentration_data(format: str): bool
_init_plots()
_init_controls()
_connect_panel_signals()
-_calculate_concentration_curve()
-_plot_concentration_data()
-_plot_curve_fit()
-_plot_prediction_interval()
-_validate_concentration_data(): bool
IPlotRenderer
+render_plot(data: ndarray, plot_type: PlotType)
+apply_styling(theme: str)
+export_plot(filename: str, format: str)
+get_plot_data(): ndarray
QGroupBox
ICalibrationMethod
+calculate_concentration(absorbance: float): float
+validate_parameters(): bool
+get_method_name(): str
+get_uncertainty(): float
CalculationController
-app: SpectrometerApp
-beer_lambert_method: BeerLambertMethod
-session_calibration_method: SessionCalibrationMethod
-transfer_model_method: TransferModelMethod
-uncertainty_calculator: UncertaintyCalculator
+__init__(app: SpectrometerApp)
+update_display_only()
+calculate_and_register()
+calculate_with_uncertainty(): Tuple[float, float]
+get_active_method(): ICalibrationMethod
+set_calculation_method(method: ICalibrationMethod)
+validate_calculation_setup(): bool
-_calculate_concentration_core(): Optional[float]
-_get_current_absorbance_spectrum(): Optional[ndarray]
-_beer_lambert_calculation(absorbance: float): float
-_session_calibration_calculation(absorbance: float): float
-_transfer_model_calculation(absorbance: float): float
-_validate_calculation_inputs(): bool
-_propagate_uncertainty(): float
IMeasurementDevice
+initialize_device(): bool
+read_measurement(): ndarray
+calibrate_device(): bool
+get_device_status(): Dict[str, Any]
+shutdown_device()
SystemInfo
+version: str = "1.0.0"
+build_date: str
+python_version: str
+qt_version: str
+numpy_version: str
+matplotlib_version: str
+dependencies: Dict[str, str]
+hardware_info: Dict[str, str]
+os_info: Dict[str, str]
+__init__()
+get_system_specifications(): Dict[str, Any]
+check_dependencies(): bool
+generate_system_report(): str
+check_hardware_compatibility(): bool
+get_performance_metrics(): Dict[str, float]
+export_system_info(filename: str): bool
-_detect_hardware()
-_check_dependency_versions()
-_measure_system_performance()
PlotOptionsState
+superimposed_spectra_labels: List[str]
+use_log_scale_y: bool
+show_markers: bool
+show_fit_on_intensity_graph: bool
+main_plot_type: PlotType
+plot_theme: str
+grid_enabled: bool
+legend_position: str
+__init__()
+reset_to_defaults()
+validate_settings(): bool
+export_settings(): Dict[str, Any]
+import_settings(settings: Dict[str, Any])
PlotWidget
-figure: Figure
-canvas: FigureCanvas
-ax: Axes
-toolbar: NavigationToolbar2QT
-layout: QVBoxLayout
-current_theme: str
-plot_renderer: IPlotRenderer
-zoom_history: List[Tuple[float, float, float, float]]
-annotation_manager: AnnotationManager
+__init__(parent: QWidget)
+clear()
+draw()
+plot(x_data: ndarray, y_data: ndarray, **kwargs): Line2D
+scatter(x_data: ndarray, y_data: ndarray, **kwargs): PathCollection
+set_xlabel(label: str)
+set_ylabel(label: str)
+set_title(title: str)
+set_xlim(xmin: float, xmax: float)
+set_ylim(ymin: float, ymax: float)
+legend(**kwargs)
+grid(visible: bool)
+apply_theme(is_dark: bool)
+get_axes(): Axes
+get_figure(): Figure
+zoom_to_region(x1: float, y1: float, x2: float, y2: float)
+reset_zoom()
+export_plot(filename: str, format: str, dpi: int): bool
+add_annotation(x: float, y: float, text: str)
+remove_annotations()
+set_interactive_mode(enabled: bool)
-_setup_plot()
-_setup_toolbar()
-_handle_mouse_events(event)
-_update_plot_appearance()
CollapsibleGroupBox
-content_widget: QWidget
-is_collapsed: bool
-toggle_button: QPushButton
-content_layout: QVBoxLayout
-main_layout: QVBoxLayout
-animation: QPropertyAnimation
-collapse_height: int
-expand_height: int
+__init__(title: str, parent: QWidget)
+toggle_collapsed()
+set_collapsed(collapsed: bool)
+add_widget(widget: QWidget)
+set_content_layout(layout: QLayout)
+is_collapsed_state(): bool
+set_animation_enabled(enabled: bool)
+set_animation_duration(duration: int)
-_setup_ui()
-_on_toggle_clicked()
-_animate_collapse()
-_animate_expand()
signal+toggled(bool)
signal+animationFinished()
BeerLambertMethod
-epsilon: float
-path_length: float
-wavelength: int
+__init__(epsilon: float, path_length: float, wavelength: int)
+calculate_concentration(absorbance: float): float
+validate_parameters(): bool
+get_method_name(): str
+get_uncertainty(): float
+set_epsilon(epsilon: float)
+set_path_length(path_length: float)
SessionCalibrationMethod
-slope: float
-intercept: float
-r_squared: float
-wavelength: int
+__init__(slope: float, intercept: float, r_squared: float, wavelength: int)
+calculate_concentration(absorbance: float): float
+validate_parameters(): bool
+get_method_name(): str
+get_uncertainty(): float
+update_calibration(slope: float, intercept: float, r_squared: float)
TransferModelMethod
-transfer_slope: float
-transfer_intercept: float
-reference_method: ICalibrationMethod
-wavelength: int
+__init__(transfer_slope: float, transfer_intercept: float, reference_method: ICalibrationMethod)
+calculate_concentration(absorbance: float): float
+validate_parameters(): bool
+get_method_name(): str
+get_uncertainty(): float
+set_reference_method(method: ICalibrationMethod)
EspectrofotometroArduino
-sensor_as726x: AS726X
-led_white: Pin
-serial_port: HardwareSerial
-led_intensity: int
-sensor_gain: int
-integration_time: int
-device_id: str
-firmware_version: str
+setup()
+loop()
+read_sensor_data(): String
+control_led(intensity: int): bool
+set_sensor_gain(gain: int): bool
+set_integration_time(time_ms: int): bool
+send_spectrum_data()
+handle_serial_commands()
+get_device_info(): String
+perform_self_test(): bool
+calibrate_sensor(): bool
+reset_device()
-_initialize_sensor(): bool
-_configure_led()
-_read_all_channels(): String
-_format_sensor_data(data: Array): String
-_parse_command(command: String): CommandStruct
-_send_response(response: String)
-_handle_error(error_code: int)
ConfigManager
-config_file: str
-default_settings: Dict[str, Any]
-current_config: Dict[str, Any]
-config_validator: ConfigValidator
+__init__(config_file: str)
+load_config(): Dict[str, Any]
+save_config(settings: Dict[str, Any]): bool
+get_setting(key: str): Any
+set_setting(key: str, value: Any)
+reset_to_defaults()
+validate_config(): bool
+export_config(filename: str): bool
+import_config(filename: str): bool
+get_config_history(): List[Dict[str, Any]]
-_apply_default_settings()
-_backup_current_config()
-_restore_config_backup()
MeasurementDataState
+calibration_data_raw: List[str]
+measurement_data_raw: List[str]
+last_calibration_values: Optional[ndarray]
+last_raw_measurement_spectrum: Optional[ndarray]
+last_measurement_values: Optional[ndarray]
+blank_subtraction_active: bool
+superimposed_spectra: Dict[str, ndarray]
+measurement_metadata: List[MeasurementMetadata]
+data_quality_history: List[QualityReport]
+__init__()
+clear_all_data()
+add_superimposed_spectrum(label: str, spectrum: ndarray)
+remove_superimposed_spectrum(label: str)
+get_current_spectrum(): Optional[ndarray]
+add_measurement_metadata(metadata: MeasurementMetadata)
+get_statistics(): Dict[str, float]
+export_all_data(): Dict[str, Any]
+validate_data_integrity(): bool
StatisticsCalculator
-calculation_cache: Dict[str, Any]
-precision_digits: int
+__init__(precision: int)
+calculate_mean(data: ndarray): float
+calculate_std_deviation(data: ndarray): float
+calculate_variance(data: ndarray): float
+calculate_snr(signal: ndarray, noise: ndarray): float
+calculate_rsd(data: ndarray): float
+calculate_median(data: ndarray): float
+calculate_percentiles(data: ndarray, percentiles: List[float]): List[float]
+detect_outliers(data: ndarray, method: str): List[int]
+apply_smoothing(data: ndarray, method: str, window_size: int): ndarray
+calculate_peak_properties(data: ndarray): Dict[str, float]
+perform_trend_analysis(data: ndarray): Dict[str, float]
+calculate_correlation(x_data: ndarray, y_data: ndarray): float
-_validate_input_data(data: ndarray): bool
-_cache_calculation(key: str, result: Any)
-_get_cached_calculation(key: str): Any
SerialProtocol
+COMMAND_READ: str = "READ"
+COMMAND_LED_ON: str = "LED_ON"
+COMMAND_LED_OFF: str = "LED_OFF"
+COMMAND_SET_GAIN: str = "SET_GAIN"
+COMMAND_SET_INTEGRATION: str = "SET_INT"
+COMMAND_CALIBRATE: str = "CALIBRATE"
+COMMAND_STATUS: str = "STATUS"
+COMMAND_RESET: str = "RESET"
+RESPONSE_OK: str = "OK"
+RESPONSE_ERROR: str = "ERROR"
+RESPONSE_DATA: str = "DATA"
+parse_command(command: str): Dict[str, Any]
+format_response(data: Any): str
+validate_checksum(data: str): bool
+calculate_checksum(data: str): str
+encode_command(command: str, parameters: Dict[str, Any]): str
+decode_response(response: str): Dict[str, Any]
-_validate_command_structure(command: str): bool
-_extract_parameters(command: str): Dict[str, Any]
HelpDialog
-content: Dict[str, str]
-text_browser: QTextBrowser
-search_widget: QLineEdit
-close_button: QPushButton
-print_button: QPushButton
-export_button: QPushButton
-layout: QVBoxLayout
-help_index: Dict[str, List[str]]
+__init__(parent: QWidget)
+show_help(section: str)
+search_help(query: str): List[str]
+load_help_content()
+update_help_content(content: Dict[str, str])
+export_help_to_pdf(filename: str): bool
+print_help_section(section: str)
-_setup_ui()
-_setup_search()
-_load_help_from_file()
-_format_help_text(text: str): str
-_build_help_index()
-_highlight_search_results(query: str)
«static»
Constants
+WAVELENGTHS: List[int] = [450, 500, 550, 570, 600, 650]
+COLORS_LIGHT: Dict[str, str]
+COLORS_DARK: Dict[str, str]
+DEFAULT_EPSILON_VALUES: Dict[int, float]
+CSS_STYLES_LIGHT: str
+CSS_STYLES_DARK: str
+SERIAL_BAUDRATE: int = 115200
+DEFAULT_PATH_LENGTH: float = 1.0
+PLOT_DPI: int = 100
+MAX_MEASUREMENT_TIME: int = 3600
+MIN_SIGNAL_THRESHOLD: float = 0.001
+MAX_NOISE_LEVEL: float = 0.1
+DEFAULT_INTEGRATION_TIME: int = 500
+CALIBRATION_UNCERTAINTY_THRESHOLD: float = 0.05
+QUALITY_SCORE_THRESHOLD: float = 0.8
DataValidator
-validation_rules: Dict[str, Callable]
-error_tolerance: float
+__init__(tolerance: float)
+validate_spectrum_data(data: ndarray): bool
+validate_concentration_value(value: float): bool
+validate_calibration_points(points: List[Tuple[float, float]]): bool
+check_data_integrity(raw_data: str): bool
+validate_wavelength_range(wavelengths: ndarray): bool
+check_measurement_consistency(measurements: List[ndarray]): bool
+validate_file_format(filename: str, expected_format: str): bool
+add_validation_rule(name: str, rule: Callable)
+remove_validation_rule(name: str)
-_check_data_range(data: ndarray, min_val: float, max_val: float): bool
-_check_data_completeness(data: ndarray): bool
-_validate_numeric_data(data: Any): bool
CalibrationState
+active_session_calibration_curve: Optional[Tuple[float, float, float]]
+active_transfer_model: Dict[int, Tuple[float, float, float]]
+reference_calibration_parameters: Dict[int, Dict[str, float]]
+epsilon_session: str
+path_length_session: str
+wavelength_for_session_calibration: int
+epsilon_values: Dict[int, float]
+calibration_history: List[Dict[str, Any]]
+calibration_uncertainty: Dict[int, float]
+__init__()
+clear_session_calibration()
+clear_transfer_model()
+set_session_calibration(slope: float, intercept: float, r_squared: float)
+set_transfer_model(wavelength: int, slope: float, intercept: float, r_squared: float)
+calculate_calibration_uncertainty(): Dict[int, float]
+validate_calibration(): bool
+export_calibration_data(): Dict[str, Any]
+import_calibration_data(data: Dict[str, Any])
SessionManager
-current_session: str
-session_data: Dict[str, Any]
-session_directory: str
-auto_save_enabled: bool
-session_metadata: MeasurementMetadata
+__init__(session_directory: str)
+start_new_session(name: str): bool
+save_session_state(): bool
+load_session_state(session_name: str): bool
+get_session_info(): Dict[str, Any]
+list_available_sessions(): List[str]
+delete_session(session_name: str): bool
+export_session(session_name: str, format: str): bool
+set_auto_save(enabled: bool, interval: int)
+get_session_statistics(): Dict[str, Any]
-_create_session_directory(session_name: str)
-_validate_session_data(data: Dict[str, Any]): bool
-_auto_save_session()
QualityController
-quality_thresholds: Dict[str, float]
-quality_history: List[QualityReport]
-max_history_size: int
+__init__(thresholds: Dict[str, float])
+assess_measurement_quality(data: ndarray): QualityReport
+check_baseline_stability(data: ndarray): bool
+validate_peak_shape(data: ndarray): bool
+check_signal_noise_ratio(data: ndarray): float
+detect_systematic_errors(data: ndarray): List[str]
+generate_quality_report(): QualityReport
+set_quality_thresholds(thresholds: Dict[str, float])
+get_quality_trends(): Dict[str, float]
+export_quality_history(filename: str): bool
-_calculate_baseline_drift(data: ndarray): float
-_analyze_peak_symmetry(data: ndarray): float
-_detect_contamination_patterns(data: ndarray): bool
-_assess_reproducibility(measurements: List[ndarray]): float
Logger
-log_file: str
-log_level: LogLevel
-max_log_size: int
-log_rotation_enabled: bool
-log_formatters: Dict[LogLevel, str]
+__init__(log_file: str, level: LogLevel)
+debug(message: str)
+info(message: str)
+warning(message: str)
+error(message: str)
+critical(message: str)
+set_log_level(level: LogLevel)
+get_log_history(): List[str]
+clear_log()
+export_log(filename: str): bool
+rotate_log_files()
-_write_log_entry(level: LogLevel, message: str)
-_format_log_message(level: LogLevel, message: str): str
-_check_log_size()
MeasurementControlState
+is_active: bool
+current_type: Optional[str]
+m_samples: int
+p_measurements: int
+current_m_count: int
+current_p_count: int
+accumulated_sub_samples_raw_lines: List[str]
+collected_final_sequential_points: List[Tuple[float, ...]]
+sequential_output_filename: Optional[str]
+sequential_initial_fit_line: Optional[str]
+sequential_subtraction_active_at_start: bool
+measurement_start_time: datetime
+estimated_completion_time: Optional[datetime]
+measurement_progress: float
+__init__()
+reset_measurement_state()
+start_new_measurement(measurement_type: str)
+is_measurement_complete(): bool
+calculate_progress(): float
+estimate_completion_time(): datetime
+get_measurement_duration(): timedelta
+validate_measurement_parameters(): bool
BackupManager
-backup_directory: str
-auto_backup_enabled: bool
-backup_interval: int
-max_backups: int
-compression_enabled: bool
+__init__(backup_directory: str)
+create_backup(include_data: bool): str
+restore_from_backup(backup_file: str): bool
+schedule_auto_backup(interval: int)
+list_available_backups(): List[str]
+verify_backup_integrity(backup_file: str): bool
+delete_old_backups(keep_count: int): int
+get_backup_info(backup_file: str): Dict[str, Any]
+set_backup_settings(settings: Dict[str, Any])
-_create_backup_archive(data_paths: List[str]): str
-_extract_backup_archive(backup_file: str, destination: str): bool
-_calculate_backup_checksum(backup_file: str): str
-_validate_backup_checksum(backup_file: str, expected_checksum: str): bool
IDataExporter
+export_data(data: Any, format: str): bool
+get_supported_formats(): List[str]
+validate_export_parameters(data: Any, format: str): bool
ErrorHandler
-error_log: List[str]
-critical_errors: List[str]
-error_callbacks: Dict[str, Callable]
-max_error_history: int
+__init__(max_history: int)
+handle_serial_error(error: Exception)
+handle_calculation_error(error: Exception)
+handle_file_error(error: Exception)
+handle_critical_error(error: Exception)
+log_error(message: str, error_type: str)
+get_error_history(): List[str]
+get_critical_errors(): List[str]
+register_error_callback(error_type: str, callback: Callable)
+clear_error_history()
+export_error_report(filename: str): bool
-_format_error_message(error: Exception): str
-_categorize_error(error: Exception): str
-_trigger_error_callbacks(error_type: str, error: Exception)
FileManager
-app: SpectrometerApp
-export_directory: str
-supported_formats: List[str]
-compression_enabled: bool
-data_exporter: IDataExporter
+__init__(app: SpectrometerApp)
+export_log(): bool
+export_plot(plot_widget: PlotWidget): bool
+export_csv_data(): bool
+export_json_data(): bool
+export_excel_data(): bool
+save_session(): bool
+load_session(): bool
+export_calibration_data(): bool
+import_calibration_data(): bool
+export_measurement_report(format: str): bool
+batch_export_data(formats: List[str]): bool
+set_export_directory(directory: str)
+get_export_statistics(): Dict[str, Any]
-_get_export_filename(extension: str): str
-_save_plot_as_image(plot_widget: PlotWidget, filename: str): bool
-_prepare_csv_data(): str
-_prepare_json_data(): str
-_prepare_excel_data(): bytes
-_save_session_data(filename: str): bool
-_load_session_data(filename: str): bool
-_validate_file_format(filename: str, format: str): bool
-_compress_data(data: bytes): bytes
-_decompress_data(data: bytes): bytes
PlotType
INTENSITY
ABSORBANCE
TRANSMITTANCE
MeasurementType
CONTINUOUS
SINGLE_AVERAGED
SEQUENTIAL
LogLevel
DEBUG
INFO
WARNING
ERROR
CRITICAL
uses
uses
uses
uses
uses
uses
uses
validates
validates
processes
calculates
exports
backups